

YZ 203 - Week 1: Lab / Practice Session Notes

Sep 22, 2026 · @murat

Lab Objectives

By the end of this session, the student will be able to:

- Verify a JDK installation from the terminal
- Create a new NetBeans project **without using the GUI Builder**
- Build a `JFrame` + `JPanel` in code and show a window on screen
- Create `JLabel`, `JButton`, `TextField` and `PasswordField` one at a time, then combine them into a working form
- Rearrange the same form with `FlowLayout`, `BorderLayout` and `GridLayout`
- Position components freely with `null` layout and `setBounds()`
- Point to one concrete example of an **interface** (`ActionListener`) and of **polymorphism** in the code just written

Duration: About 2 class hours (the hands-on block after the theory lecture) **Required**

materials: JDK installed, NetBeans (or your preferred IDE)

Environment Setup: JDK and NetBeans

1. JDK Installation

1. Download Eclipse Temurin (Adoptium) or Oracle JDK — a current LTS release
2. Complete the installer with default settings
3. Open a terminal or CMD and verify with:

```
java -version
javac -version
```

4. If both print a version number, the install succeeded. If a student gets an error, the JDK's install directory may be missing from PATH — check and add it if needed

2. NetBeans Installation

1. Download the current release from netbeans.apache.org

2. Run the installer (it usually detects the JDK path automatically)
3. Open NetBeans: **File → New Project → Java with Ant → Java Application**
4. Enter a project name (e.g. `Week1_Intro`); leave **"Create Main Class"** checked

Ground rule: The GUI Builder (drag-and-drop design tool) is not used in this course. Every component is written by hand in the Source view. Students should avoid accidentally switching to the Design tab and dragging components in.

Exercise 1: First JFrame and JPanel

Goal: Open an empty window in code — no components yet.

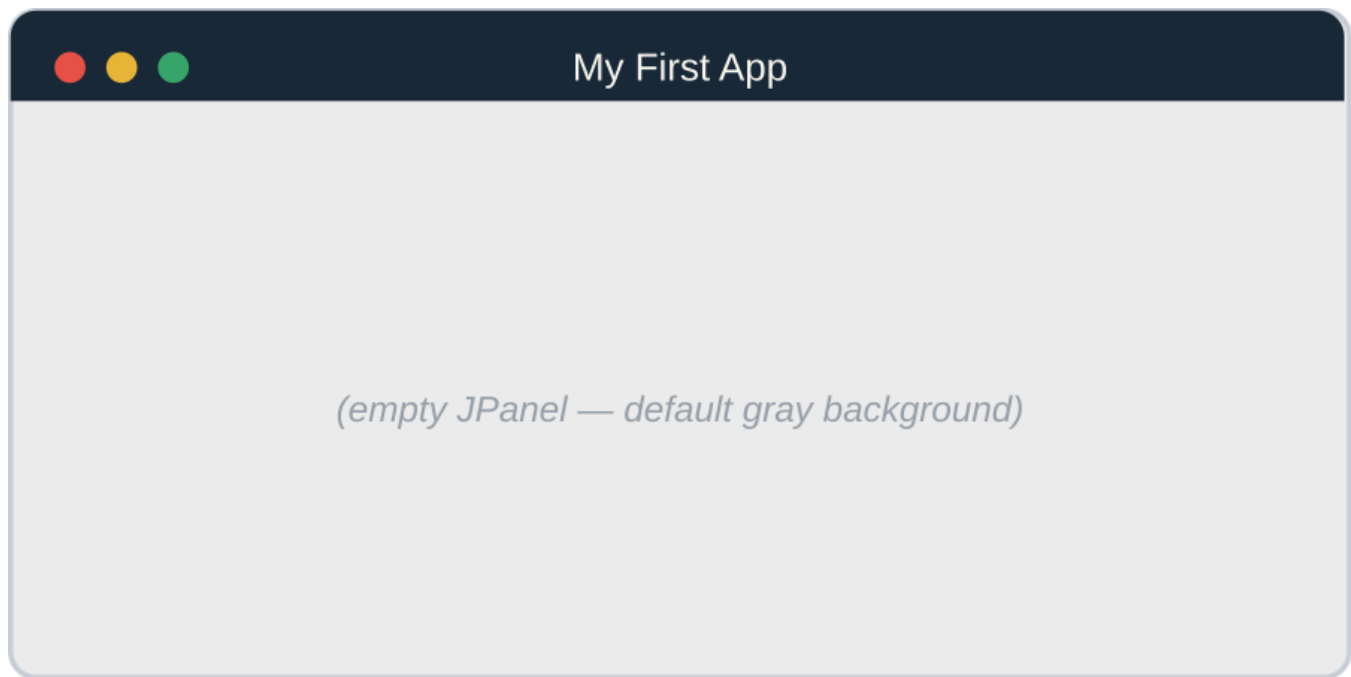
1. In the `Week1_Intro` project, create a new `.java` file (e.g. `FirstWindow.java`)
2. Type the following line by line together with the class; run it after each new line and observe the change:

```
import javax.swing.*;

public class FirstWindow {
    public static void main(String[] args) {
        JFrame frame = new JFrame("My First App");
        JPanel panel = new JPanel();

        frame.add(panel);
        frame.setSize(400, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Expected screen output:



3. Students should try: what happens if the `setSize` line is removed? What happens if `setVisible(true)` is moved to the very top?

Discussion point: why `setVisible(true)` is always called last — the window is assembled first, then shown.

Exercise 2: JLabel on Its Own

Goal: Display text with a `JLabel` — confirm that it never reacts to clicks or typing.

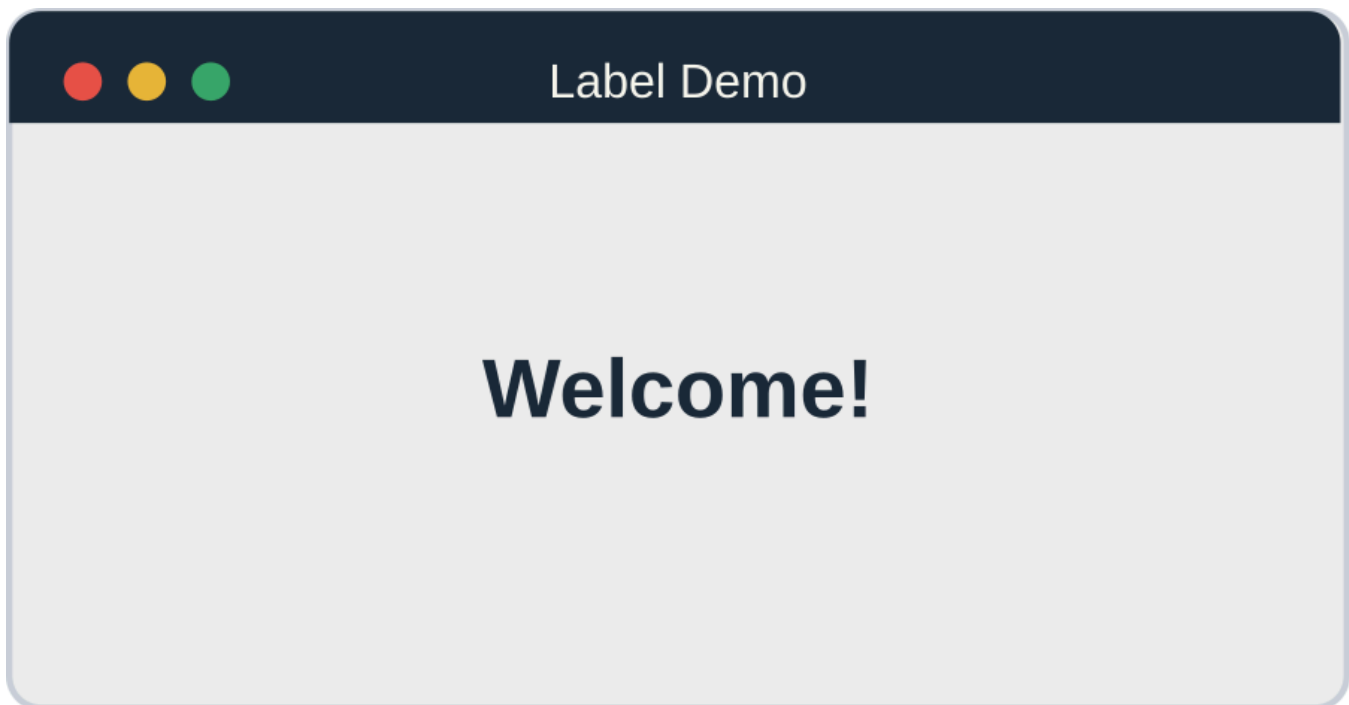
```
import javax.swing.*;

public class LabelDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Label Demo");
        JPanel panel = new JPanel();

        JLabel title = new JLabel("Welcome!");
        title.setFont(new java.awt.Font("Arial", java.awt.Font.BOLD, 20));

        panel.add(title);
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Expected screen output:



Try it: call `title.setText("Updated!")` from a second line right after creating it — the label's text changes before the window is even shown, since nothing has happened yet at that point.

Exercise 3: JButton on Its Own

Goal: Fire an event with a `JButton` and an `ActionListener` — see the label update on click.

```
import javax.swing.*;

public class ButtonDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Button Demo");
        JPanel panel = new JPanel();

        JLabel status = new JLabel("Ready");
        JButton clickMe = new JButton("Click me");

        clickMe.addActionListener(e -> status.setText("Clicked!"));

        panel.add(status);
        panel.add(clickMe);
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Expected screen output — before and after the click:



Try it: add a counter variable (`int[] count = {0};`) and set the label text to `"Clicked " + (++count[0]) + " times"` on every click.

Exercise 4: JTextField on Its Own

Goal: Read what the user typed with `getText()` and use it in another component.

```
import javax.swing.*;

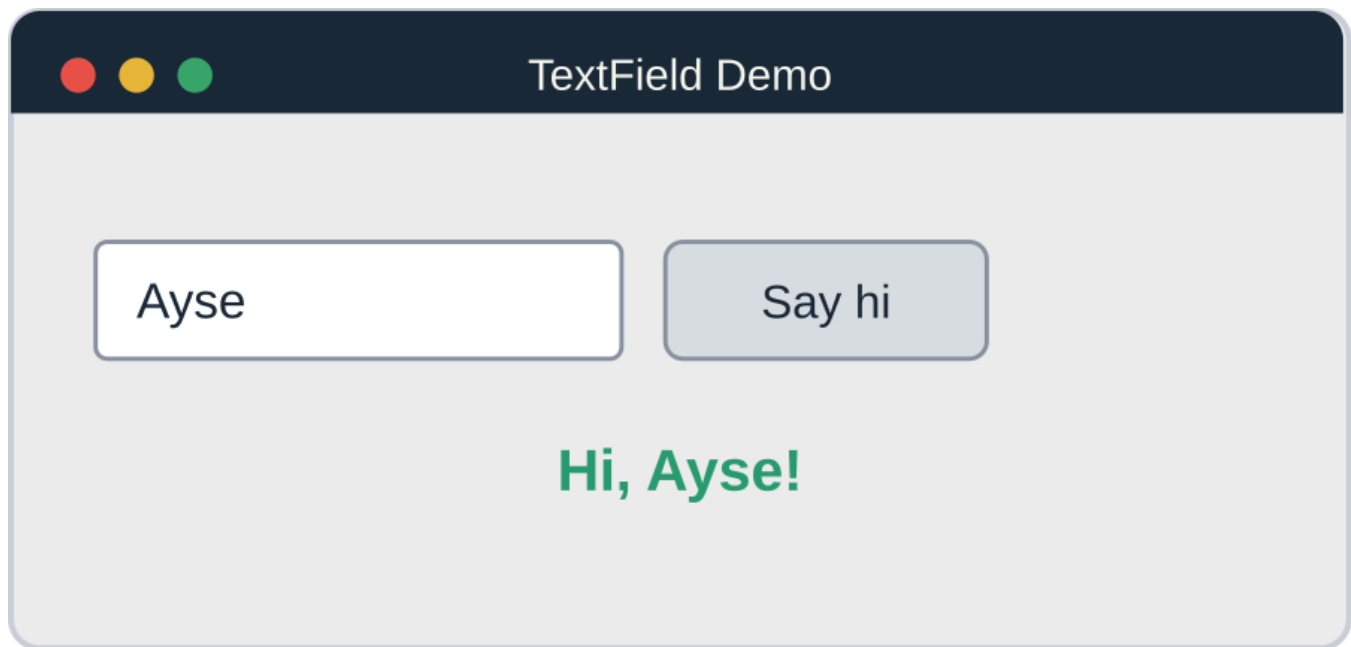
public class TextFieldDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("TextField Demo");
        JPanel panel = new JPanel();

        JTextField nameField = new JTextField(12);
        JButton sayHi = new JButton("Say hi");
        JLabel greeting = new JLabel(" ");

        sayHi.addActionListener(e -> {
            String name = nameField.getText();
            greeting.setText("Hi, " + name + "!");
        });

        panel.add(nameField);
        panel.add(sayHi);
        panel.add(greeting);
        frame.add(panel);
        frame.setSize(320, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Expected screen output (after typing "Ayse" and clicking):



Try it: what happens if the field is left empty and the button is clicked? Add a check: `if (name.isEmpty()) greeting.setText("Please type a name");`

Exercise 5: JPasswordField on Its Own

Goal: See the masking behaviour and use `getPassword()`, which returns a `char[]`, never a `String`.

```
import javax.swing.*;

public class PasswordDemo {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Password Demo");
        JPanel panel = new JPanel();

        JPasswordField pwField = new JPasswordField(12);
        JButton check = new JButton("Check length");
        JLabel result = new JLabel(" ");

        check.addActionListener(e -> {
            char[] pwd = pwField.getPassword();
            result.setText("Length: " + pwd.length);
        });

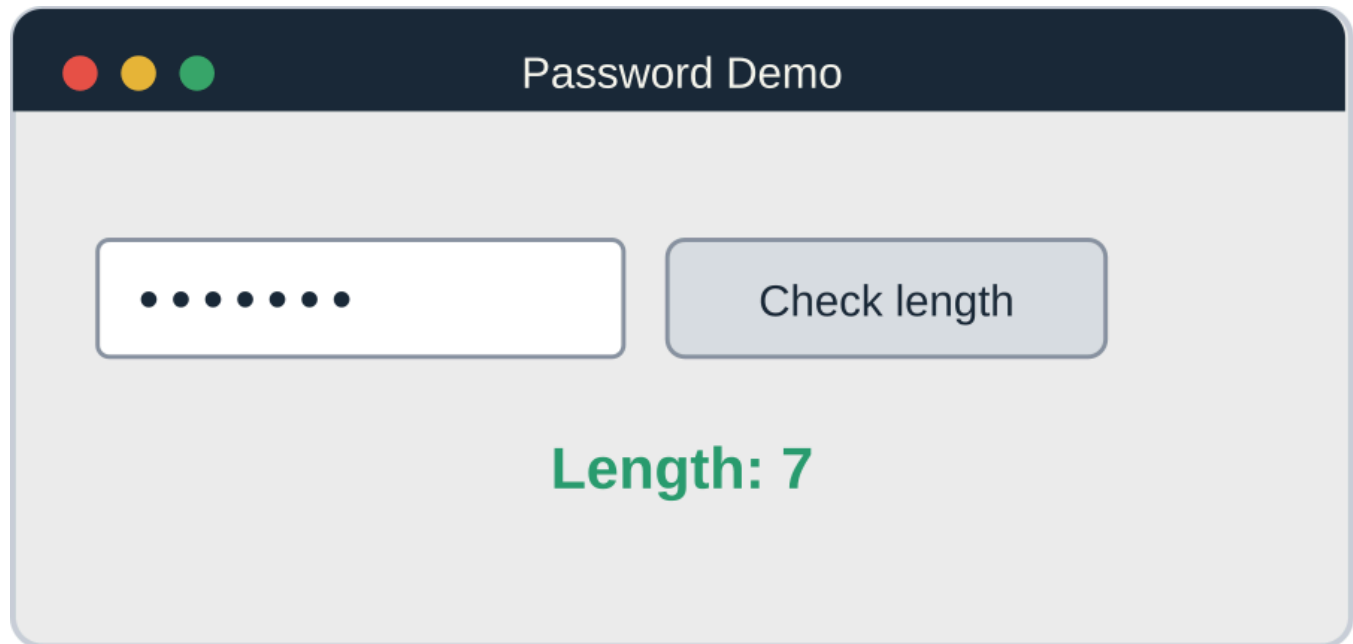
        panel.add(pwField);
        panel.add(check);
    }
}
```

```

        panel.add(result);
        frame.add(panel);
        frame.setSize(320, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}

```

Expected screen output (after typing a 7-character password):



Discussion point: why `getPassword()` returns `char[]` and not `String` — a `String` stays in memory until garbage collected, while a `char[]` can be zeroed out right after use for better security.

Exercise 6: Combining All Four into a Login Form

Goal: Put `JLabel`, `TextField`, `JPasswordField` and `Button` on one panel and wire a single `ActionListener` that reads two inputs and writes to a third component.

```

import javax.swing.*;

public class LoginForm {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Login Form");
        JPanel panel = new JPanel();
    }
}

```

```

JLabel userLabel = new JLabel("Username:");
JTextField userField = new JTextField(12);
JLabel pwLabel = new JLabel("Password:");
JPasswordField pwField = new JPasswordField(12);
JButton loginBtn = new JButton("Log in");
JLabel statusLabel = new JLabel(" ");

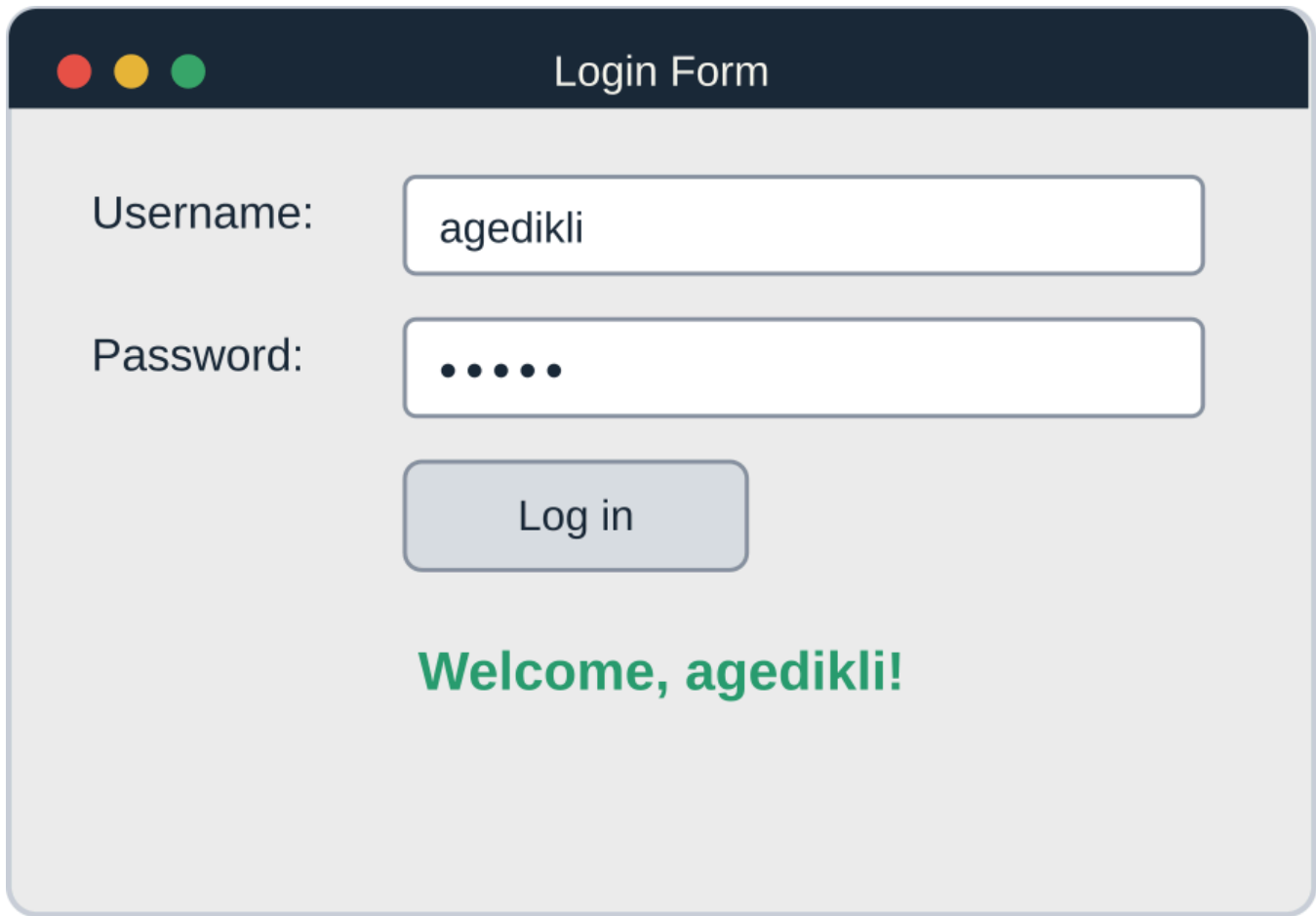
loginBtn.addActionListener(e -> {
    String user = userField.getText();
    char[] pwd = pwField.getPassword();
    if (user.isEmpty() || pwd.length == 0) {
        statusLabel.setText("Fill in both fields");
    } else {
        statusLabel.setText("Welcome, " + user + "!");
    }
});

panel.add(userLabel);
panel.add(userField);
panel.add(pwLabel);
panel.add(pwField);
panel.add(loginBtn);
panel.add(statusLabel);

frame.add(panel);
frame.setSize(320, 220);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Expected screen output (after typing a username, a password, and clicking Log in):



Try it: if either field is left empty, click Log in and confirm the label shows "Fill in both fields" instead of a welcome message.

Observation: components are laid out left to right in the order they are added to the panel — that is `FlowLayout`, `JPanel` 's default (covered in the next exercise).

Exercise 7: Rearranging the Same Form with Different Layouts

Goal: Take the form from Exercise 6 and change only `panel.setLayout(...)` to see the layout manager's visual effect directly.

7a. FlowLayout (observe the default)

```
panel.setLayout(new FlowLayout());
```

Expected screen output (narrow window — components wrap to a second row):

Username:

Password:

Welcome!

(narrow window: items wrap to the next row)

Resize the window narrower and wider — watch components wrap onto a new row.

7b. BorderLayout

```
panel.setLayout(new BorderLayout());  
panel.add(userLabel, BorderLayout.NORTH);  
panel.add(userField, BorderLayout.CENTER);  
panel.add(loginBtn, BorderLayout.SOUTH);
```

Expected screen output:

Username: NORTH

agedikli

CENTER (stretches)

Log in SOUTH

Note: adding a second component to the same region (e.g. `CENTER`) makes the new one replace the old — this is a common beginner mistake, worth demonstrating live.

7c. GridLayout

```
panel.setLayout(new GridLayout(4, 2, 8, 8));
panel.add(userLabel);
panel.add(userField);
panel.add(pwLabel);
panel.add(pwField);
```

Expected screen output:

Username: agedikli

Password:

(every cell is forced to the same size)

`GridLayout(4, 2, 8, 8)` : 4 rows, 2 columns, 8px horizontal/vertical gap between cells — every cell is the same size.

Try it: switch through all three layouts in a row and write one sentence noting the visual difference between them.

Exercise 8: Free Positioning with null Layout

Goal: Turn off the layout manager and position components by pixel coordinates — a first try at the approach used later (Week 5 onward) for the game project.

```
import javax.swing.*;

public class FreePosition {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Free Positioning");
        JPanel panel = new JPanel();
        panel.setLayout(null);

        JButton button1 = new JButton("Button 1");
        button1.setBounds(50, 30, 100, 40);
    }
}
```

```
        JButton button2 = new JButton("Button 2");  
        button2.setBounds(180, 90, 100, 40);  
  
        panel.add(button1);  
        panel.add(button2);  
  
        frame.add(panel);  
        frame.setSize(400, 300);  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
        frame.setVisible(true);  
    }  
}
```

Expected screen output:



`setBounds(x, y, width, height)`: `x / y` is the top-left corner's coordinate inside the panel; `width / height` is the component's size.

Try it: move the second button to the window's bottom-right corner and the first to the top-left, finding the coordinates by trial and error.

Discussion point: why `null` layout is not "always the way to go" (components stay fixed in place when the window is resized instead of scaling with it) — but why it is preferred for game-like, motion-based interfaces.

Behind the Scenes: Interface and Polymorphism

Every exercise above used `addActionListener(e -> ...)`. Under the hood, `ActionListener` is a Java **interface** with one method, `actionPerformed(ActionEvent e)`. The lambda `e -> statusLabel.setText(...)` is one way of implementing it; an older, more explicit way is:

```
button.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        statusLabel.setText("Clicked!");
    }
});
```

Both compile, and both are valid `ActionListener` implementations.

`addActionListener(ActionListener)` never needs to know which style was used — it only ever calls `actionPerformed()`. One interface, many implementations, used interchangeably: that is **polymorphism**, and it is the exact mechanism the mid-semester game project will lean on for keyboard input and the game loop.

Submission Checklist

- ☐ `java -version` and `javac -version` run successfully
- ☐ NetBeans project created without using the GUI Builder
- ☐ `FirstWindow.java` opens an empty window
- ☐ `LabelDemo.java`, `ButtonDemo.java`, `TextFieldDemo.java`, `PasswordDemo.java` each run and match their expected output
- ☐ `LoginForm.java` combines all four components and validates empty fields
- ☐ The login form was tried with `FlowLayout`, `BorderLayout` and `GridLayout`
- ☐ `FreePosition.java` places two buttons at chosen corners with `setBounds()`
- ☐ Can point to `ActionListener` as an interface and explain the polymorphism in `addActionListener(...)`

Next Week

Weeks 2-3 go deeper into basic components with many more examples (`JCheckBox` , `JRadioButton` , `ButtonGroup` , `JComboBox` , `JTextArea` , `JList`); from Week 5 on, the class moves to animation with `Timer` and starts building the egg-catching game.